Exercício: Generics - O Cesto de Peixes e Polvos

Objetivo

O objetivo deste exercício é entender e aplicar conceitos de **classes genéricas** em Java. Você irá criar classes para representar diferentes tipos de objetos e um cesto genérico que pode armazenar qualquer tipo de objeto.

Instruções

1. Crie duas classes para representar diferentes tipos de animais marinhos.
2. Crie uma classe genérica que pode armazenar qualquer tipo de objeto.
3. Implemente métodos para guardar e pegar objetos do cesto.
4. Teste a funcionalidade do cesto genérico com instâncias das classes de animais marinhos.

Passos para Fazer o Exercício

1. **Defina as classes dos animais marinhos**:
   - Crie uma classe `Peixe` com atributos `nome` e `tamanho`.
   - Crie uma classe `Polvo` com atributos `nome` e `tentaculos`.
2. **Crie a classe genérica `Cesto`**:
   - Defina a classe `Cesto` com um atributo genérico `conteudo`.
   - Implemente métodos `guardar` e `pegar` para armazenar e recuperar objetos do cesto.
3. **Teste a funcionalidade**:
   - No método `main`, crie instâncias de `Cesto` para `Peixe` e `Polvo`.
   - Guarde instâncias de `Peixe` e `Polvo` nos respectivos cestos.
   - Recupere e imprima os objetos armazenados.

Código do Exercício

```
// Classe Peixe
class Peixe {
    private String nome;
    private double tamanho;

    // Construtor
    public Peixe(String nome, double tamanho) {
        this.nome = nome;
        this.tamanho = tamanho;
    }

    // Getters
    public String getNome() {
        return nome;
    }

    public double getTamanho() {
        return tamanho;
    }
}

// Classe Polvo
class Polvo {
    private String nome;
    private int tentaculos;

    // Construtor
    public Polvo(String nome, int tentaculos) {
        this.nome = nome;
        this.tentaculos = tentaculos;
    }

    // Getters
    public String getNome() {
        return nome;
    }

    public int getTentaculos() {
        return tentaculos;
    }
}

// Cesto genérico
class Cesto<T> {
    private T conteudo;

    public void guardar(T item) {
        conteudo = item;
    }
```

```
    public T pegar() {
        return conteudo;
    }
}

// Exemplo de uso
public class Main {
    public static void main(String[] args) {
        Cesto<Peixe> cestoPeixes = new Cesto<>();
        cestoPeixes.guardar(new Peixe("Dourado", 30.5));
        Peixe peixe = cestoPeixes.pegar();
        System.out.println("Peixe guardado: " + peixe.getNome() + ",
Tamanho: " + peixe.getTamanho());

        Cesto<Polvo> cestoPolvos = new Cesto<>();
        cestoPolvos.guardar(new Polvo("Octopus", 8));
        Polvo polvo = cestoPolvos.pegar();
        System.out.println("Polvo guardado: " + polvo.getNome() + ",
Tentáculos: " + polvo.getTentaculos());
    }
}
```

Explicação do Código

1. **Classes Peixe e Polvo**:
    - Ambas as classes possuem atributos específicos (`nome`, `tamanho` para `Peixe` e `nome`, `tentaculos` para `Polvo`).
    - Construtores são usados para inicializar os objetos.
    - Métodos getters são fornecidos para acessar os atributos.
2. **Classe Genérica Cesto**:
    - A classe `Cesto` é genérica e pode armazenar qualquer tipo de objeto.
    - O método `guardar` armazena um objeto no cesto.
    - O método `pegar` retorna o objeto armazenado.
3. **Classe Main**:
    - Instâncias de `Cesto` são criadas para `Peixe` e `Polvo`.
    - Objetos `Peixe` e `Polvo` são armazenados e recuperados dos cestos.
    - As informações dos objetos armazenados são impressas no console.

Exercício: Usando Generics para Manipular Preços e Letras

Objetivo:

O objetivo deste exercício é praticar a criação e uso de uma classe genérica chamada `Caixa`.

Instruções:

1. Crie uma classe chamada `Caixa<T>` que seja genérica, onde `T` representa o tipo de elemento que a caixa pode armazenar.
2. A classe `Caixa` deve ter os seguintes métodos:

- `public Caixa(int capacidade)`: Construtor que inicializa a capacidade da caixa.
- `public void adicionar(T elemento, int indice)`: Adiciona um elemento à caixa na posição especificada pelo índice.
- `public T obter(int indice)`: Retorna o elemento da caixa na posição especificada pelo índice.

3. No método `main`, crie duas instâncias da classe `Caixa`:

- Uma para armazenar preços (do tipo `Double`).
- Outra para armazenar letras (do tipo `Character`).
    - Observação: A classe `Character` nos permite trabalhar com caracteres (char) de forma mais, de forma semelhante a que fizemos com Integer durante as aulas para manipular números inteiros (int).

4. Adicione elementos às caixas e imprima os resultados conforme o exemplo do código fornecido.

Passos para fazer o exercício:

1. Implemente a classe `Caixa<T>` com os métodos especificados.
2. Crie uma instância da classe `Caixa<Double>` para armazenar preços.
3. Adicione alguns preços à caixa de preços.
4. Crie outra instância da classe `Caixa<Character>` para armazenar letras.
5. Adicione algumas letras à caixa de letras.
6. Imprima os elementos das caixas conforme o exemplo do código original.

Código do Exercício:

```
public class Caixa<T> {
    private T[] elementos;

    public Caixa(int capacidade) {
        elementos = (T[]) new Object[capacidade];
    }

    public void adicionar(T elemento, int indice) {
        elementos[indice] = elemento;
    }

    public T obter(int indice) {
        return elementos[indice];
    }

    public static void main(String[] args) {
        // Criando uma CaixaDePreços para Double
        Caixa<Double> caixaDePrecos = new Caixa<>(3);
        caixaDePrecos.adicionar(10.5, 0);
        caixaDePrecos.adicionar(20.0, 1);

        System.out.println("Preço 1: " + caixaDePrecos.obter(0)); //
Saída: 10.5
        System.out.println("Preço 2: " + caixaDePrecos.obter(1)); //
Saída: 20.0

        // Criando uma CaixaDeLetras para Character
        Caixa<Character> caixaDeLetras = new Caixa<>(2);
        caixaDeLetras.adicionar('A', 0);
        caixaDeLetras.adicionar('B', 1);

        System.out.println("Letra 1: " + caixaDeLetras.obter(0)); //
Saída: A
        System.out.println("Letra 2: " + caixaDeLetras.obter(1)); //
Saída: B
    }
}
```

Explicação do Código:

- A classe `Caixa<T>` é genérica e permite armazenar elementos de qualquer tipo.
- O construtor `Caixa(int capacidade)` inicializa o array de elementos com a capacidade especificada.
- Os métodos `adicionar` e `obter` permitem adicionar e recuperar elementos da caixa, respectivamente.
- No método `main`, criamos duas instâncias da classe `Caixa` para armazenar preços e letras, demonstrando o uso da classe genérica.